Taller: Componentes Front-End con React

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Este taller responde las cuatro preguntas de la guía sobre React, el framework con el que está construida la interfaz web de **MONK GRAIN**. Los ejemplos no son genéricos: salen del código del proyecto, de modo que cada concepto se puede ir a ver funcionando.

2. Objetivo

Explicar la diferencia entre React y JSX, qué son las clases en React, cuáles son sus principales eventos, y representar sus conceptos en un mapa, junto con las características principales del framework.

3. Características principales de React

Característica	Qué significa en la práctica
Basado en componentes	La interfaz se arma con piezas que se combinan. En MONK GRAIN, `CoverageBar` muestra cuánto se cubrió de una solicitud, y la misma pieza sirve al comprador y al vendedor
Declarativo	Se describe qué debe verse según el estado, no cómo llegar ahí paso a paso. No se escribe «borra esta fila y añade aquella»: se dice cómo se ve la lista y React calcula la diferencia
Flujo de datos en un solo sentido	Los datos bajan de padre a hijo por propiedades. Un hijo no modifica lo que recibe: avisa con una función. Así, cuando algo se ve mal, se busca hacia arriba y no en todas partes
DOM virtual	React mantiene una representación en memoria, la compara con la anterior y toca el documento real solo donde cambió. Tocar el DOM es caro; compararlo en memoria, no
Hooks	Funciones que dan estado y efectos a un componente sin necesidad de clases, y que permiten extraer lógica repetida. En este proyecto, `useProtectedResource` encapsula lo que 28 pantallas repetían

4. Diferencia entre React y JSX

Es la confusión más común, y la respuesta corta es que **no son cosas comparables**: React es una biblioteca y JSX es una sintaxis.

	React	JSX
Qué es	Una biblioteca de JavaScript para construir interfaces	Una extensión de sintaxis que permite escribir marcado dentro de JavaScript
Qué aporta	Componentes, estado, hooks, el DOM virtual y la reconciliación	Solo una forma más legible de escribir la estructura
¿Es obligatorio?	Sí, si se quiere usar React	**No.** React funciona sin JSX
¿El navegador lo entiende?	Sí, es JavaScript	No. Hay que transformarlo antes

4.1 La demostración

Esto es JSX, tomado de `CoverageBar`:

```
<div className="mt-1 h-1.5 w-full rounded-full bg-muted">
  <div style={{ width: `${percentage}%` }} />
</div>
```

Y esto es **exactamente lo mismo**, escrito sin JSX:

```
React.createElement(
  "div",
  { className: "mt-1 h-1.5 w-full rounded-full bg-muted" },
  React.createElement("div", { style: { width: percentage + "%" } })
);
```

El compilador —Babel, o el de Next.js— convierte el primero en el segundo antes de que el navegador vea nada. **JSX es azúcar sintáctico sobre** `React.createElement`: desaparece al compilar y no llega al navegador.

> **Por qué existe de todos modos.** Las dos versiones hacen lo mismo, pero una estructura de seis niveles escrita con `createElement` es ilegible: los paréntesis se acumulan y no se ve qué contiene qué. JSX conserva la forma de árbol que tiene el resultado.

4.2 Lo que JSX no es

No es HTML, aunque se le parezca. Las diferencias importan porque son errores frecuentes:

En HTML	En JSX	Por qué
`class`	`className`	`class` es palabra reservada de JavaScript
`for`	`htmlFor`	Igual: `for` es un bucle
`onclick="..."`	`onClick={...}`	En JSX son camelCase y reciben una función, no una cadena
`style="width: 50%"`	`style={{ width: "50%" }}`	Un objeto de JavaScript, no texto
Etiquetas sin cerrar	Toda etiqueta se cierra	JSX debe poder convertirse en una llamada a función

5. Las clases en React

Hasta la versión 16.8 de React, un componente que necesitara **estado** o reaccionar a su **ciclo de vida** tenía que ser una clase. Una función no podía recordar nada entre renders.

```
class ContadorDePosturas extends React.Component {
  constructor(props) {
    super(props);
    this.state = { posturas: 0 };          // el estado vive en this.state
    this.sumar = this.sumar.bind(this);    // había que atar el `this`
  }

  componentDidMount() {                    // al montarse
    this.cargar();
  }

  componentDidUpdate(prevProps) {          // al cambiar algo
    if (prevProps.id !== this.props.id) this.cargar();
  }

  componentWillUnmount() {                 // al desmontarse
    this.cancelar();
  }

  sumar() {
    this.setState({ posturas: this.state.posturas + 1 });
  }

  render() {                               // obligatorio: lo que se ve
    return <p>{this.state.posturas} posturas</p>;
  }
}
```

Elemento	Para qué
`extends React.Component`	Hereda el comportamiento de un componente
`this.state`	Los datos que, al cambiar, vuelven a dibujar
`this.setState()`	La **única** forma de cambiarlo. Asignar directamente no redibuja
`this.props`	Lo que recibe del padre. Es de solo lectura
`render()`	Devuelve lo que se ve. Es obligatorio
Métodos de ciclo de vida	`componentDidMount`, `componentDidUpdate`, `componentWillUnmount`

5.1 Por qué hoy casi no se usan

Los hooks dieron a las funciones lo que antes solo tenían las clases. El mismo componente de arriba, en la forma que usa MONK GRAIN:

```
function ContadorDePosturas({ id }) {
  const [posturas, setPosturas] = useState(0);

  useEffect(() => {
    cargar();
    return () => cancelar();   // esto reemplaza a componentWillUnmount
  }, [id]);                     // y esta lista, a componentDidUpdate

  return <p>{posturas} posturas</p>;
}
```

Con clases	Con hooks
`this.state` y `setState`	`useState`
`componentDidMount`	`useEffect(fn, [])`
`componentDidUpdate`	`useEffect(fn, [deps])`
`componentWillUnmount`	Lo que devuelve el `useEffect`

Las tres ventajas que lo decidieron: desaparece el problema del `this`, que obligaba a atar cada método; la lógica de una misma preocupación queda junta en lugar de repartida entre tres métodos de ciclo de vida; y se puede **extraer a un hook propio** y reutilizar, que es lo que hace `useProtectedResource` en este proyecto.

> **Las clases no están obsoletas.** React las sigue soportando y hay código que las usa. Pero en MONK GRAIN no hay ninguna: los 67 componentes son funciones.

6. Principales eventos de React

React envuelve los eventos del navegador en un **evento sintético**, que se comporta igual en todos los navegadores. Se nombran en camelCase y reciben una función, no una cadena de texto.

6.1 Eventos de ratón

Evento	Cuándo ocurre	En el proyecto
`onClick`	Al pulsar	Retirar una postura, aceptar, rechazar
`onDoubleClick`	Al pulsar dos veces	—
`onMouseEnter` / `onMouseLeave`	Al entrar o salir el puntero	Resaltar una fila

6.2 Eventos de formulario

Evento	Cuándo ocurre	En el proyecto
`onChange`	Al cambiar el valor de un campo	Los kilos del carrito, los filtros del catálogo
`onSubmit`	Al enviar un formulario	Publicar una solicitud, registrar la cuenta bancaria
`onFocus` / `onBlur`	Al entrar o salir del campo	Validar al salir

6.3 Eventos de teclado

Evento	Cuándo ocurre
`onKeyDown`	Al pulsar una tecla. Es el que sirve para atajos
`onKeyUp`	Al soltarla

6.4 Dos detalles que son errores frecuentes

Evitar el envío por omisión. Un formulario recarga la página si no se lo impide:

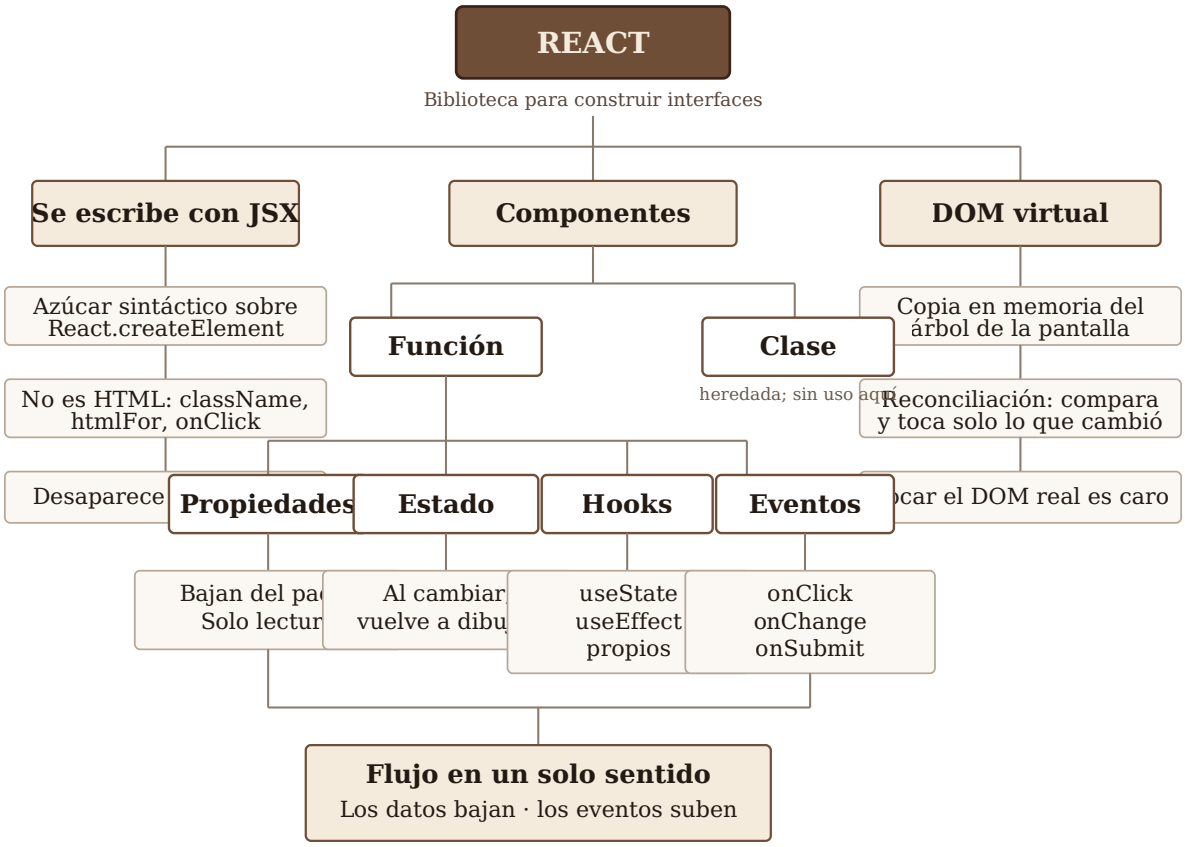
```
async function handleSave(event: React.FormEvent) {
  event.preventDefault();   // sin esto, la página se recarga y se pierde todo
  ...
}
```

Pasar la función, no llamarla. Es el error que más desconcierta a quien empieza:

```
<button onClick={withdraw(offer)}>      {/* MAL: se ejecuta al dibujar */}
<button onClick={() => withdraw(offer)}> {/* BIEN: se ejecuta al pulsar */}
```

En el primero, `withdraw(offer)` se evalúa mientras React construye la pantalla, y lo que recibe `onClick` es su resultado. La postura se retira sola, sin que nadie pulse nada.

7. Mapa conceptual de React



Mapa conceptual de React, con los conceptos del taller y su relación

8. Conclusión

React y JSX no compiten: React es la biblioteca que mantiene la interfaz al día con el estado, y JSX solo la forma legible de escribir la estructura, que desaparece al compilar. Las clases fueron la única manera de tener estado hasta 2019 y siguen funcionando, pero los hooks resolvieron sus tres problemas —el `this`, la lógica repartida entre métodos de ciclo de vida, y la imposibilidad de reutilizarla—.

Esa última ventaja no es teórica en este proyecto: la evidencia AA4-EV03 extrae un hook propio que elimina un bloque repetido en 28 pantallas.